UVM Interview Questions

Q1: What is UVM? What is the advantage of UVM?

Ans: UVM (Universal Verification Methodology) is a standardized methodology for verifying the both complex & simple digital design in simple way.

UVM Features:

First methodology & second collection of class libraries for Automation
Reusability through testbench
Plug & Play of verification IPs
Generic Testbench Development
Vendor & Simulator Independent
Smart Testbench i.e. generate legal stimulus as from pre-planned coverage plan
Support CDV –Coverage Driven Verification
Support CRV –Constraint Random Verification
UVM standardized under the Accellera System Initiative
Register modeling

Q2: UVM derived from which language?

Ans: Here is the detailed connection between SV, UVM, OVM and other methodologies.

Q3. What is the difference between uvm_component and uvm_object?

OR

We already have uvm_object, why do we need uvm_component which is actually derived class of uvm_object?

Ans:

uvm_component:

Quasi Static Entity (after build phase it is available throughout the simulation)
Always tied to a given hardware(DUT Interface) Or a TLM port
Having phasing mechanism for control the behavior of simulation
Configuration Component Topology

uvm_object:

Dynamic Entity (create when needed, transfer from one component to other & then dereference)
Not tied to a given hardware or any TLM port
Not phasing mechanism

Q4: Why phasing is used? What are the different phases in uvm?

Ans: UVM Phases is used to control the behavior of simulation in a systematic way & execute in a sequential ordered to avoid race condition. This could also be done in system verilog but manually.

List of UVM Phases:
buid_phase
connect_phase
end_of_elaboration_phase
start_of_simulation_phase
run _phase (task)
Sub Phases of Reset Phase:
pre_reset_phase
reset_phase
post_reset_phase
pre_configure_phase
configure_phase
post_configure_phase
pre_main_phase
main_phase
post_main_phase
pre_shutdown_phase
shutdown_phase
post_shutdown_phase
extract_phase
check_phase
report_phase
Below figure makes it more clear

Q5: Which uvm phase is top - down , bottom – up & parallel?
Ans: Only build phase is a top-down & other phases are bottom-up except run phase which is parallel. The build phase works top-down since the testbench hierarchy may be configure so we need to build the branches before leafs

Q6: Why build phase is top – down & connect phase is bottom – up?
Ans: The connect phase is intended to be used for making TLM connections between components, which is why it occur after build phase. It work bottom-up so that its got the correct implementation all the way up the design hierarchy, if worked top-down this would be not possible

Q7: Which phase is function & which phase is task?
Ans: Only run phase is a task (time consuming phase) & other phases are functions (non-blocking)

Q8: How uvm phases initiate?
Ans: UVM phases initiate by calling run_test(“test1”) in top module. When run_test() method call, it first create the object of test top & then call all phases.

Q7: How test cases run from simulation command line?

Ans: In top module write run_test(); i.e. Don't give anything in argument.
Then in command line : +UVM_TESTNAME=testname

Q8: Difference between module & class based TB?
Ans: A module is a static object present always during of the simulation.
A Class is a dynamic object because they can come and go during the life time of simulation.

Q9: What is uvm_config_db ? What is difference between uvm_config_db & uvm_resource_db?
Ans: uvm_config_db is a parameterized class used for configuration of different type of parameter into the uvm database, So that it can be used by any component in the lower level of hierarchy.

uvm_config_db is a convenience layer built on top of uvm_resource_db, but that convenience is very important. In particular, uvm_resource_db uses a "last write wins" approach. The uvm_config_db, on the other hand, looks at where things are in the hierarchy up through end_of_elaboration, so "parent wins." Once you start start_of_simulation, the config_db becomes "last write wins."

All of the functions in uvm_config_db#(T) are static, so they must be called using the :: operator
It is extended from the uvm_resource_db#(T), so it is child class of uvm_resource_db#(T)

Q10:What is uvm_transaction, uvm_seq_item, uvm_object, uvm_component?
Q11:What is the advantage of `uvm_component_utils() and `uvm_object_utils() ?
Q12:What is the difference between `uvm_do and `uvm_ran_send?
diff between uvm_transaction and uvm_seq_item?
Q13:What is the difference between uvm _virtual_sequencer and uvm_sequencer ?
Q14:What are the benefits of using UVM?
Q15:What is super keyword? What is the need of calling super.build() and super.connect()?
Q16:Is uvm is independent of systemverilog ?
Q17:Can we have user defined phase in UVM?
Q18:What is p_sequencer ?
Q19:What is uvm RAL model ? why it is required ?
Q20:What is the difference between new() and create?
Q21:What is analysis port?
Q22:What is TLM FIFO?
Q23:How sequence starts?
Q24:What is the difference between UVM RAL model backdoor write/read and front door write/read ?

Q25:What is objection?
Q26:What is the advantage of `uvm_pre_body and `uvm_post_body ?
Q27:What is the difference between Active mode and Passive mode?
Q28:What is the difference between copy and clone?
Q29:What is UVM factory?
Q30:What are the types of sequencer? Explain each?
Q31:What are the different phases of uvm_component? Explain each?
Q32:How set_config_* works?
Q33:hat are the advantages of uvm RAL model ?
Q34:What is the different between set_config_* and uvm_config_db ?
Q35:What are the different override types?
Q36:What is virtual sequence and virtual sequencer?
Q37:Explain end of simulation in UVM?
Q38:How to declare multiple imports?
Q39:What is symbolic representation of port, export and analysis port?
Q40:What is the difference in usage of $finish and global stop request in UVM?
Q41:Why we need to register class with uvm factory?
Q42:can we use set_config and get_config in sequence ?
Q43:What is uvm_heartbeat ?
Q44:how to access DUT signal in uvm_component/uvm_object ?

AHB Interview Questions

1. How AHB is pipelined architecture?
2. What is the size of max data can be transferred in single transfer?
3. Explain 1k boundary concept in AHB?
4. Okay response is single cycle? but error/split/retry is two cycle, why?
5. Explain the concept of two cycle response?
6. What if the slave gets the address out of range?
7. How to connect multiple slaves to single master?
8. Explain the round robin arbitration concept?
9. Explain the split-retry concept?
10. What is the difference between HREADY and HREADY_OUT signals?
11. What is the slave response for BUSY transfer?
12. What is the difference between WRAP4 and INCR4?
13. How to terminate the INCR type transfer?
14. What is difference between BURST and Beat?
15. How to calculate the size of the burst?
16. Is HREADY is Input or output to/from the slave?
17. What is align and un-align concept?
18. Explain wrapping calculation?
19. Is early burst termination is done by Slave/Arbiter?
20. Explain the LOCKED transfer?
21. What is default Master?

22. What is little-endian and big-endian?
23. How slave will detects the end of INCR type burst transfer?

AXI Interview Questions

1. How AXI is different from AHB?
2. Explain the concept of AXI 4KB boundary condition?
3. Explain the valid ready handshake in AXI?
4. Explain the channel concept?
5. Explain the out-of-order concept?
6. What is fixed burst type?
7. Explain the AXI response types?

SOC Verification Interview Questions

1. What is the difference between SOC and IP Verification?
2. Write a block diagram of SOC architecture?
3. What is the target of verification in SOC verification?
4. Is coverage is considered in SOC Verification?
5. What are the major components in SOC architecture?
6. What are the challenges of SOC verification?

ASIC Verification Interview Questions

1.What is the difference between SOC and IP Verification?
2What is the multi clock domain design?
Consider the simple memory model and explain the possible Verification scenarios?
When will you consider that verification is done?
What is the difference between IP and VIP?
Which is best among IP level and SOC level verification?
How important is code reviews?

Q :What is the difference between VHDL and Verilog?
Ans: Fundamentally speaking, not a lot. You can produce robust designs and comprehensive test environments with both languages, for both ASIC and FPGA. However, the two languages approach the task from different directions; VHDL, intended as a specification language, is very exact in its nature and hence very verbose. Verilog, intended as a simulation language, it much closer to C in style, in

that it is terse and elegant to write but requires much more care to avoid nasty bugs. VHDL doesn't let you get away with much; Verilog assumes that whatever you wrote was exactly what you intended to write. If you get a VHDL architecture to compile, it's probably going to approximate to the function you wanted. For Verilog, successful compilation merely indicates that the syntax rules were met, nothing more. VHDL has some features that make it good for system-level modeling, whereas Verilog is much better than VHDL at gate-level simulation.

What is latch up in CMOS design and ways to prevent it?

A Problem which is inherent in the p-well and n-well processes is due to relatively large number of junctions which are formed in these structures, the consequent presence of parasitic diodes and transistors.

Latch-up is a condition in which the parasitic components give rise to the Establishment of low resistance conducting path between VDD and VSS with Disastrous results

Latch-up may be induced by glitches on the supply rails or by incident radiation.

Latch-up pertains to a failure mechanism wherein a parasitic thyristor (such as a parasitic silicon controlled rectifier, or SCR) is inadvertently created within a circuit, causing a high amount of current to continuously flow through it once it is accidentally triggered or turned on. Depending on the circuits involved, the amount of current flow produced by this mechanism can be large enough to result in permanent destruction of the device due to electrical overstress (EOS).

Preventions for Latch-Up

by adding tap wells, for example in an Inverter for NMOS add N+ tap in n-well and connect it to Vdd, and for PMOS add P+ tap in p-substrate and connect it to Vss.
an increase in substrate doping levels with a consequent drop in the value of Rs.
reducing Rp by control of fabrication parameters and by ensuring a low contact resistance to Vss.
and the other is by introducing of guard rings.....
Latchup in Bulk CMOS

A byproduct of the Bulk CMOS structure is a pair of parasitic bipolar transistors. The collector of each BJT is connected to the base of the other transistor in a positive feedback structure. A phenomenon called latchup can occur when (1) both BJT's conduct, creating a low resistance path between Vdd and GND and (2) the product of the gains of the two transistors in the feedback loop, b1 x b2, is greater than one. The result of latchup is at the minimum a circuit malfunction, and in the worst case, the destruction of the device.

parasitic_transitor_in_bulk_cmos Cross section of parasitic transistors in Bulk CMOS

parasitic_transitor_in_bulk_cmos_equivalent_circuit Equivalent Circuit

Latchup may begin when Vout drops below GND due to a noise spike or an improper circuit hookup (Vout is the base of the lateral NPN Q2). If sufficient current flows through Rsub to turn on Q2 (I Rsub > 0.7 V), this will draw current through Rwell. If the voltage drop across Rwell is high enough, Q1 will also turn on, and a self-sustaining low resistance path between the power rails is formed. If the gains are such that b1 x b2 > 1, latchup may occur. Once latchup has begun, the only way to stop it is to reduce the current below a critical level, usually by removing power from the circuit.

The most likely place for latchup to occur is in pad drivers, where large voltage transients and large currents are present.

Preventing latchup

Fab/Design Approaches:

Reduce the gain product b1 x b1
move n-well and n+ source/drain farther apart increases width of the base of Q2 and reduces gain beta2 > also reduces circuit density
buried n+ layer in well reduces gain of Q1
2. Reduce the well and substrate resistances, producing lower voltage drops

· higher substrate doping level reduces Rsub

· reduce Rwell by making low resistance contact to GND

· guard rings around p- and/or n-well, with frequent contacts to the rings, reduces the parasitic resistances.

cmos_transitor_with_guard_rings CMOS transistors with guard rings

Systems Approaches:

Make sure power supplies are off before plugging a board. A "hot plug in" of an unpowered circuit board or module may cause signal pins to see surge voltages greater than 0.7 V higher than Vdd, which rises more slowly to is peak value. When the chip comes up to full power, sections of it could be latched.
Carefully protect electrostatic protection devices associated with I/O pads with guard rings. Electrostatic discharge can trigger latchup. ESD enters the circuit

through an I/O pad, where it is clamped to one of the rails by the ESD protection circuit. Devices in the protection circuit can inject minority carriers in the substrate or well, potentially triggering latchup.

Radiation, including x-rays, cosmic, or alpha rays, can generate electron-hole pairs as they penetrate the chip. These carriers can contribute to well or substrate currents.

Sudden transients on the power or ground bus, which may occur if large numbers of transistors switch simultaneously, can drive the circuit into latchup. Whether this is possible should be checked through simulation.